



Photo by jggrz on pixabay

Lecture 12

—

GMM & EM

C-Falter (*Polygonia c-album*)

- ♦ Seinen ungewöhnlichen Namen verdankt der C-Falter seinem einzigartigen Erkennungsmerkmal: Während die Oberseite des Falters orange-braun gemustert ist, zeichnet sich auf der Unterseite der Hinterflügel ein markantes, weißes „C“ ab.
- ♦ In Deutschland ist der C-Falter recht häufig zu finden. Dabei fühlt er sich besonders an feuchten Standorten wohl, aber auch Parkanlagen und Gärten besucht er gerne. Er erreicht eine Flügelspannweite von 40-50mm.
- ♦ Da er mit Vorliebe den Nektar von Beerensträuchern trinkt, gilt der C-Falter als äußerst guter Bestäuber für diese.



Photo by Daniel Wanke on pixabay

Quellen:

* [https://de.wikipedia.org/wiki/Admiral_\(Schmetterling\)](https://de.wikipedia.org/wiki/Admiral_(Schmetterling))

* <https://www.infranken.de/ratgeber/tiere/10-der-schoensten-schmetterlinge-in-deutschland-praechtig-und-bedroht-art-5250577>



Clustering – GMM & EM

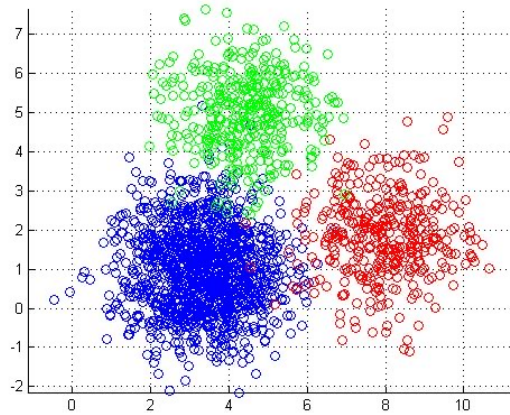
1. From k-means to GMM
2. One-Dimensional GMM and EM
3. Multidimensional GMM and EM
4. Maximum Likelihood Estimation for GMM



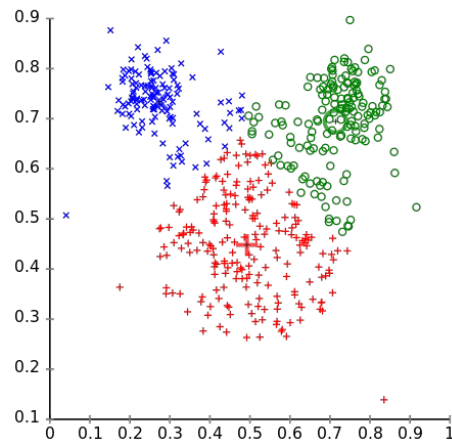
K-means to Gaussian Mixture Models (GMM) (1)

◆ K-means

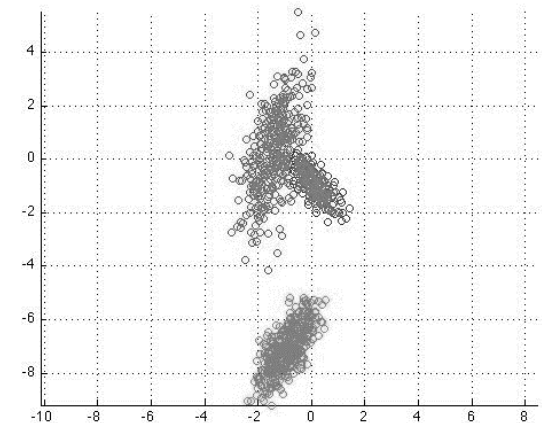
- Each Cluster modelled by cluster center
- Each data point assigned to nearest Cluster (using Euclidean distance) → „hard clustering“
- Limitations
 - What about overlapping clusters?
 - What about non-circular clusters?



source: <https://de.mathworks.com/>



source: https://en.wikipedia.org/wiki/K-means_clustering



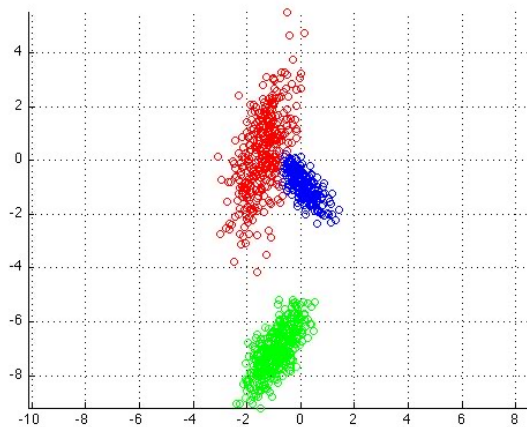
source: <https://de.mathworks.com/>



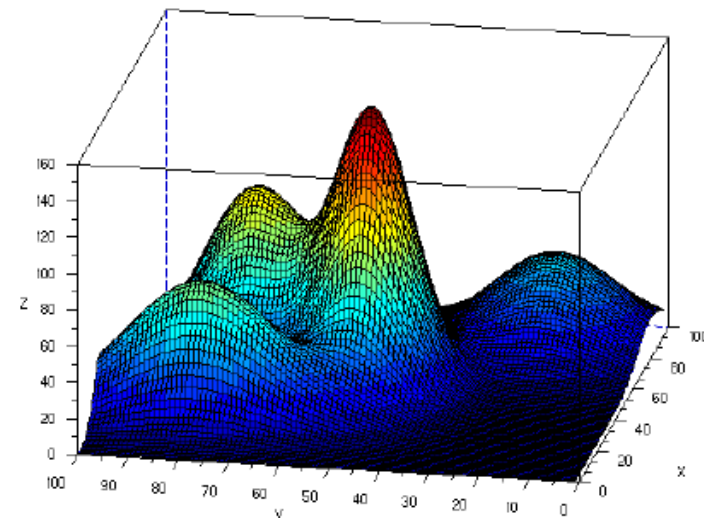
K-means to Gaussian Mixture Models (GMM) (2)

♦ Gaussian Mixture Model (GMM)

- models each cluster as a probability distribution, not just a centroid
- assigns each data point to each Cluster with a probability → „soft clustering“
- EM-Algorithm → infers mean / (co)variance / weight of each cluster
i.e. how many (%) data points does each cluster contain
(does not have to be evenly distributed)



source: <https://de.mathworks.com/>



source: https://www.researchgate.net/figure/224105715_fig1_Fig-3-3D-view-of-a-4D-Gaussian-Mixture-Model-used-in-our-experiments



Gaussian Mixture Model

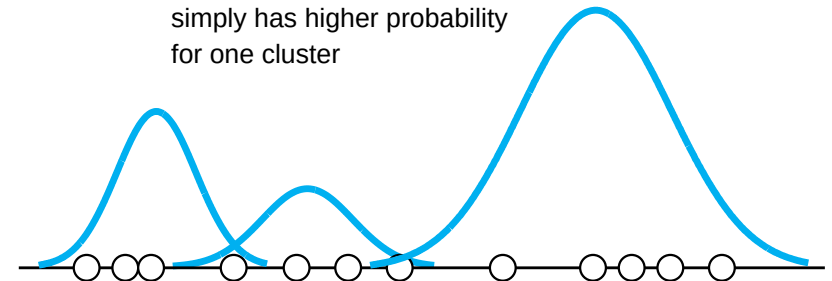
- ♦ Model k clusters as Gaussian distributions with
 - mean μ_c
 - variance σ_c^2 and
 - weight π_c for each cluster c (the percentage of data points generated by c with $\sum_{i=1}^k \pi_i = 1$)
- ♦ Joint probability distribution: weighted sum $p(x) = \sum_{c=1}^k \pi_c \mathcal{N}(x; \mu_c, \sigma_c^2)$

with $\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$ < one-dim gaussian distribution

- ♦ „Latent“ form:
 - 1) select mixture component with probability π : $p(z = c) = \pi_c$
 - 2) draw from this components Gaussian: $p(x | z = c) = \mathcal{N}(x; \mu_c, \sigma_c^2)$

➔ Latent variable z : we observe x , but z is hidden from us

each data point *will* have a probability for each cluster, so we don't 100% know what cluster it belongs to
simply has higher probability for one cluster





Clustering – GMM & EM

1. From k-means to GMM
2. One-Dimensional GMM and EM
3. Multidimensional GMM and EM
4. Maximum Likelihood Estimation for GMM



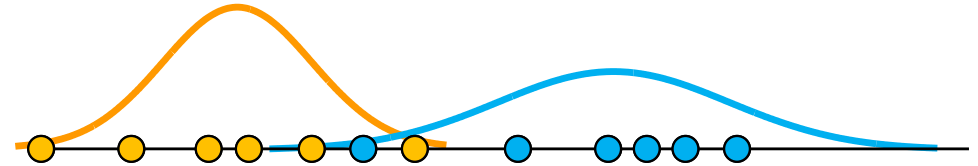
One-dimensional Mixture Models

Given $x_1, x_2, \dots, x_n$ data points

Assumptions 1:

- $k = 2$ Gaussians with unknown parameters μ and σ^2
- we know which Gaussians each data points comes from

→ $\mu_b = \frac{\sum_{i=1}^{n_b} x_i}{n_b}$ and $\sigma_b^2 = \frac{\sum_{i=1}^{n_b} (x_i - \mu_b)^2}{n_b}$ so we could calculate μ and σ

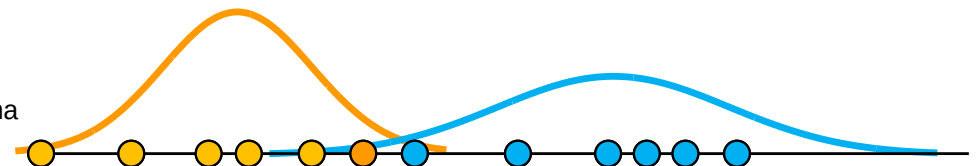
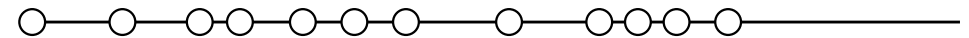


Assumption 2:

- we know the parameters μ and σ^2
- unknown which data points come from which Gaussian

→ $P(b | x_i) = \frac{P(x_i|b)P(b)}{P(x_i|b)P(b) + P(x_i|a)P(a)}$ (Bayes Rule) with

$P(x_i|b) = \mathcal{N}(x_i; \mu_b, \sigma_b^2)$ could calc gaussian from μ and σ



→ **Chicken and Egg Problem**

if we know one, we could calculate the other but we don't know either



The EM Algorithm in 1-dim (1)

♦ Chicken and Egg Problem

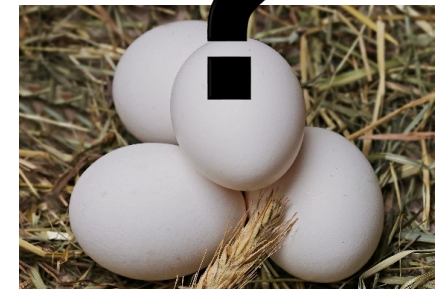
- Need the parameters of the Gaussians to guess the origin of each data point
- Need the origin of each data point to estimate (max. likelihood) the parameters of the Gaussians

♦ Solution: The Expectation-Maximization (EM) Algorithm

- Idea: Start with a random initialization and iterate essentially like k-means

Notation:

- number of clusters k (given)
- mean μ_c , variance σ_c^2 and weight π_c for each cluster c
- probability estimation r_{ic} that data point i belongs to cluster c





The EM Algorithm in 1-dim (2)

weight must sum up to 1

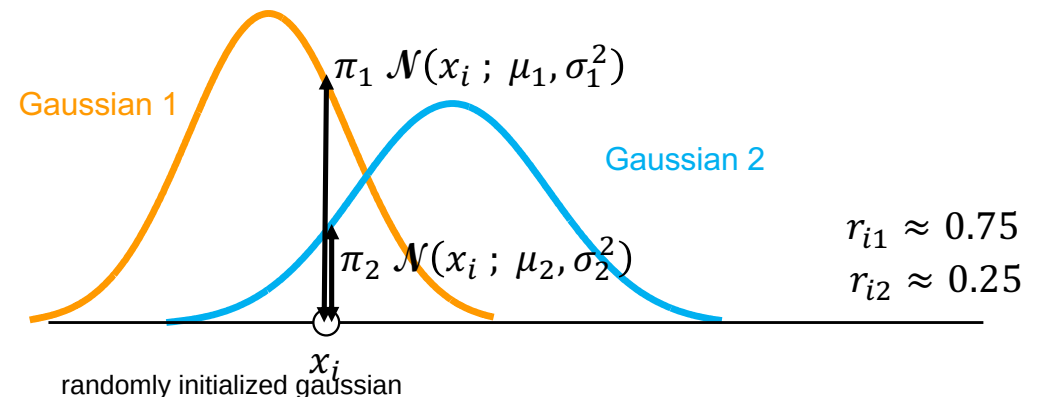
- ♦ Initialization: randomly choose mean μ_c , variance σ_c^2 and weight π_c for each cluster c

Iterate the E- and M-steps until convergence:

- ♦ **E-Step**: update the probability estimation (responsibility) r_{ic} that x_i belongs to cluster c

$$r_{ic} = \frac{\pi_c \mathcal{N}(x_i; \mu_c, \sigma_c^2)}{\sum_{c'=1}^k \pi_{c'} \mathcal{N}(x_i; \mu_{c'}, \sigma_{c'}^2)}$$

- probability that data point x_i belongs to cluster c
- divide by normalization to make the r 's sum to 1 for each x_i





The EM Algorithm in 1-dim (3)

♦ **M-Step:** Maximization: update parameters μ_c , σ_c^2 and weight π_c for each cluster c

■ Responsibility of each cluster c :

$$m_c = \sum_{i=1}^n r_{ic} \quad \text{sums all } r \text{ of one cluster}$$

■ Weight of each cluster c :

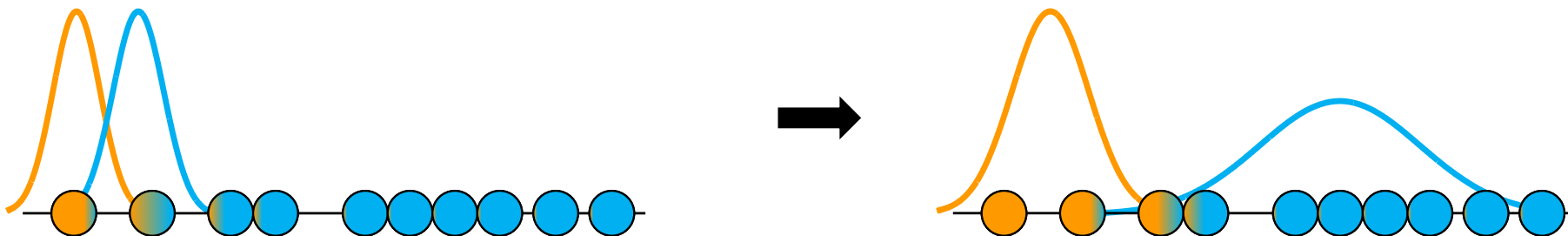
$$\pi_c = \frac{m_c}{\sum_{c'=1}^k m_{c'}} \quad \text{i.e. normalize}$$

■ Weighted mean of each cluster c :

$$\mu_c = \frac{1}{m_c} \sum_{i=1}^n r_{ic} x_i$$

■ Weighted variance of each cluster c :

$$\sigma_c^2 = \frac{1}{m_c} \sum_{i=1}^n r_{ic} (x_i - \mu_c)^2 \quad (\text{using the new } \mu_c !)$$



would then iterate further with E-Step and M-Step in sequence
re-calc expectation (cf. above) which essentially moves the gaussian centered around the given points,
> so works like k-means but instead of centroid, gaussian is moved



EM - Convergence

◆ Each step increases the (log)likelihood of our model

this value will always increase, meaning the gaussians can never move back and return to a previous model

$$L = \log(P(x_1, x_2, \dots, x_n)) = \log(\prod_{i=1}^n P(x_i)) = \sum_{i=1}^n \log(\sum_{c=1}^k \pi_c \mathcal{N}(x_i; \mu_c, \sigma_c^2))$$

thus, it is guaranteed that the model will converge to one model that is most likely

Proof: see e.g. https://en.wikipedia.org/wiki/Expectation%E2%80%93maximization_algorithm

◆ Convergence

- guaranteed monotonic improvement of likelihood breaking point is min difference of current L to prev L

- stop iteration when L (or the parameters) stop changing very much

- no guarantee of global optimum: may end up in local optimum

- initialization important → run multiple times, use L to choose best result
with different random inits

adv: can compare L, so models are comparable to find best global optimum

◆ Hard assignments: assign to most likely cluster

◆ New points (out-of-training-sample): assign via posterior probabilities

problem: larger k leads to higher likelihood but we want smallest possible k
bc each data point would get a higher cluster probability due to "tighter" clusters
but does not actually represent correct clusters



Choosing the number of clusters with BIC

♦ Problem

- If we increase the number of clusters k , the likelihood usually improves
- But more clusters also mean:
 - more parameters
 - higher model complexity
 - greater risk of overfitting

♦ Bayesian Information Criterion (BIC): balances model fit & model complexity

$$BIC = -2 \cdot \log L + p \cdot \log n$$

L : maximized likelihood of the model

p : total number of free model parameters

n : number of data points

p stays the same within the same dimensionality
 $p = 3k - 1$ for 1-dim

Note: p depends on k , the dimension d and the covariance structure

→ lower BIC = better trade-off between fit and complexity

♦ How to use BIC

- fit GMMs with different values of k
- compute the BIC for each model
- choose the model with the lowest BIC



Exercises

Exercise 1

1-d EM in Python





Clustering – GMM & EM

1. From k-means to GMM
2. One-Dimensional GMM and EM
3. Multidimensional GMM and EM
4. Maximum Likelihood Estimation for GMM



EM for multivariate (d -dim) GMMs (1)

- ◆ Data points are d -dimensional vectors: $\vec{x}_1, \vec{x}_2, \dots, \vec{x}_n$
- ◆ Mean $\vec{\mu}_c$ is a d -dim vector: $\vec{\mu}_c = \frac{1}{n} \sum_{i=1}^n \vec{x}_i$
- ◆ „Variance“ is a $d \times d$ -dim Covariance matrix: $\hat{\Sigma}_c = \frac{1}{n} \sum_{i=1}^n (\vec{x}_i - \vec{\mu}_c)^T (\vec{x}_i - \vec{\mu}_c)$
- ◆ Multivariate Gaussian distribution: $\mathcal{N}(\vec{x}; \vec{\mu}, \hat{\Sigma}) = \frac{1}{\sqrt{(2\pi)^d \det(\hat{\Sigma})}} \exp\left(-\frac{1}{2} (\vec{x} - \vec{\mu})^T \hat{\Sigma}^{-1} (\vec{x} - \vec{\mu})\right)$

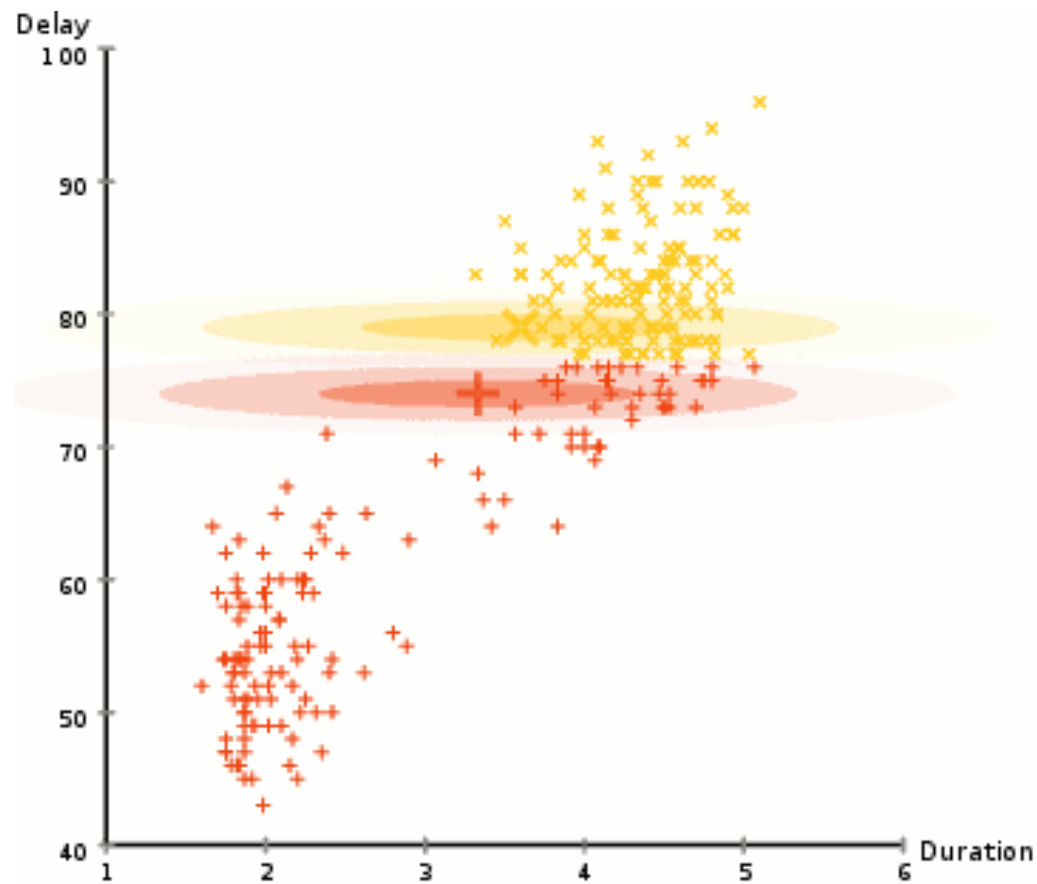


EM for multivariate (d -dim) GMMs (2)

- ◆ E-Step: $r_{ic} = \frac{\pi_c \mathcal{N}(\vec{x}_i; \vec{\mu}_c, \hat{\Sigma}_c)}{\sum_{c'} \pi_{c'} \mathcal{N}(\vec{x}_i; \vec{\mu}_{c'}, \hat{\Sigma}_{c'})}$ (same but with multivariate Gaussian)
- ◆ M-Step:
 - „Responsibility“ of each cluster c : $m_c = \sum_{i=1}^n r_{ic}$ (unchanged)
 - Weight of each cluster c : $\pi_c = \frac{m_c}{\sum_{c'=1}^k m_{c'}}$ (unchanged)
 - Weighted mean of each cluster c : $\vec{\mu}_c = \frac{1}{m_c} \sum_{i=1}^n r_{ic} \vec{x}_i$ (same in vector form)
 - Weighted covariance of each cluster c : $\hat{\Sigma}_c = \frac{1}{m_c} \sum_{i=1}^n r_{ic} (\vec{x}_i - \vec{\mu}_c)(\vec{x}_i - \vec{\mu}_c)^T$



2-dim EM Demo

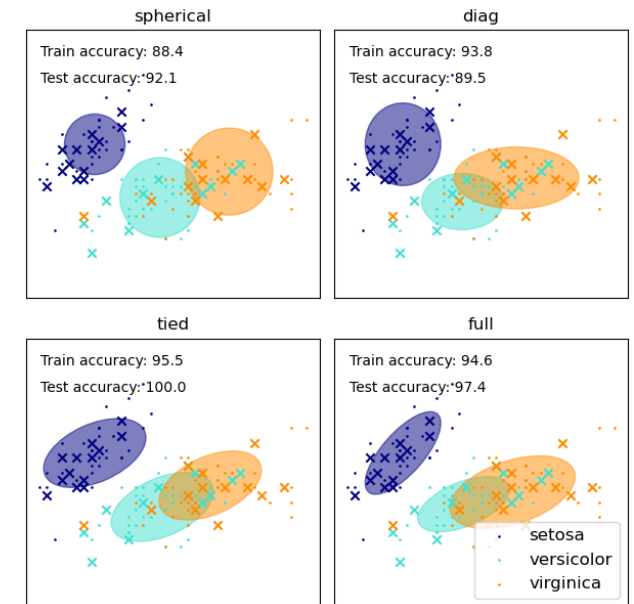


source: <https://github.com/lovasoa/expectation-maximization>



Covariance Structures and Parameter Counts in GMM

- ◆ Covariance structure determines covariance matrix is symmetrical
 - how flexible each Gaussian cluster is
 - how many parameters the model must estimate
- ◆ A d-dimensional GMM has
 - one mean vector per cluster: d parameters per cluster
 - mixture weights: k-1 free parameters for the whole model
 - one covariance per cluster: depends on the chosen form
- ◆ Common covariance structures
 - **Spherical**: cluster shape: same spread in all directions
 - parameters per cluster: 1 > just k
 - **Diagonal**: cluster shape: axis-aligned ellipse / ellipsoid
 - parameters per cluster: d > k*d
 - **Full**: arbitrary symmetric covariance matrix. Cluster shape: arbitrary ellipse / ellipsoid, including rotation
 - parameters per cluster: $d*(d+1)/2$ > mult by k
 - **Tied**: all clusters share the same covariance matrix/shape (but have different means)
 - parameters for the whole model: $d*(d+1)/2$ all clusters essentially look the same
- ◆ More flexible covariance structures can fit the data better, but increase model complexity



but must still be an ellipse

source: https://scikit-learn.org/stable/auto_examples/mixture/plot_gmm_covariances.html



Exercises

Exercise 2

2d-EM Visualization





Clustering – GMM & EM

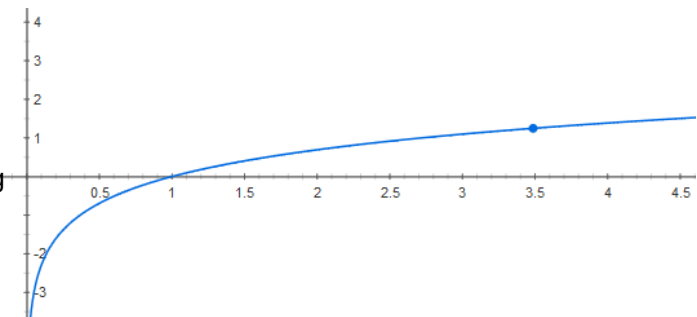
1. From k-means to GMM
2. One-Dimensional GMM and EM
3. Multidimensional GMM and EM
4. Maximum Likelihood Estimation for GMM



Estimating the parameters of a 1-d Gaussian Model from observations (1)

- ◆ Given $x_1, x_2, \dots, x_n$ data points (observations) from a 1-d Gaussian Model, estimate the *most likely* parameters μ and σ^2 of this Gaussian model
- ◆ Def: **likelihood** of $x_1, x_2, \dots, x_n$ under the model μ and σ^2 : $p(\{x_1, x_2, \dots, x_n\}|\mu, \sigma^2)$
- ◆ Maximum Likelihood Estimation (MLE): find $\hat{\mu}, \hat{\sigma} = \operatorname{argmax}_{\mu, \sigma} (p(\{x_1, x_2, \dots, x_n\}|\mu, \sigma^2))$
- ◆ Assume independence of observations: $p(\{x_1, x_2, \dots, x_n\}|\mu, \sigma^2) = \prod_{i=1}^n p(x_i|\mu, \sigma^2)$
- ◆ Take the natural log (monotonically increasing) and use Gaussian

$$\begin{aligned}\hat{\mu}, \hat{\sigma} &= \operatorname{argmax}_{\mu, \sigma} (\prod_{i=1}^n p(x_i|\mu, \sigma^2)) \\ &= \operatorname{argmax}_{\mu, \sigma} (\ln(\prod_{i=1}^n p(x_i|\mu, \sigma^2))) \quad \text{In kürzt exp später raus} \\ &= \operatorname{argmax}_{\mu, \sigma} \sum_{i=1}^n \ln(p(x_i|\mu, \sigma^2)) \quad \text{annahme: normalverteilung} \\ &= \operatorname{argmax}_{\mu, \sigma} \sum_{i=1}^n \ln\left(\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i-\mu)^2}{2\sigma^2}\right)\right)\end{aligned}$$





Estimating the parameters of a 1-d Gaussian Model from observations (2)

- ◆ Some simple math....

$$\begin{aligned}\hat{\mu}, \hat{\sigma} &= \operatorname{argmax}_{\mu, \sigma} \sum_{i=1}^n \ln\left(\frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2}\right)\right) \\ &= \operatorname{argmax}_{\mu, \sigma} \sum_{i=1}^n \left[\ln(1) - \ln(\sqrt{2\pi}\sigma) - \frac{(x_i - \mu)^2}{2\sigma^2} \right] \\ &= \operatorname{argmax}_{\mu, \sigma} \sum_{i=1}^n \left[-\frac{(x_i - \mu)^2}{2\sigma^2} - \ln(\sigma) - \ln(\sqrt{2\pi}) \right] \\ &= \operatorname{argmax}_{\mu, \sigma} \sum_{i=1}^n \left[-\frac{(x_i - \mu)^2}{2\sigma^2} - \ln(\sigma) \right] \\ &= \operatorname{argmin}_{\mu, \sigma} \sum_{i=1}^n \left[\frac{(x_i - \mu)^2}{2\sigma^2} + \ln(\sigma) \right]\end{aligned}$$

within argmax/min it is allowed to just leave out constant if they don't change the maximum



Find the minimum by taking the partial derivatives and setting to 0:

♦ For $\hat{\mu}$:

$$\begin{aligned}\frac{\partial \sum_{i=1}^n \left[\frac{(x_i - \mu)^2}{2\sigma^2} + \ln(\sigma) \right]}{\partial \mu} &= 0 \\ \sum_{i=1}^n \left[\frac{-2(x_i - \mu)}{2\sigma^2} \right] &= 0 \\ \sum_{i=1}^n [x_i - \mu] &= 0 \\ \sum_{i=1}^n x_i - n\mu &= 0 \\ \hat{\mu} &= \frac{1}{n} \sum_{i=1}^n x_i\end{aligned}$$

♦ For $\hat{\sigma}^2$:

$$\begin{aligned}\frac{\partial \sum_{i=1}^n \left[\frac{(x_i - \hat{\mu})^2}{2\sigma^2} + \ln(\sigma) \right]}{\partial \sigma} &= 0 \\ \sum_{i=1}^n \left[\frac{-2(x_i - \hat{\mu})^2}{2\sigma^3} + \frac{1}{\sigma} \right] &= 0 \\ - \sum_{i=1}^n \left[\frac{(x_i - \hat{\mu})^2}{\sigma^3} \right] + \frac{n}{\sigma} &= 0 \\ - \sum_{i=1}^n [(x_i - \hat{\mu})^2] + n\sigma^2 &= 0 \\ \hat{\sigma}^2 &= \frac{1}{n} \sum_{i=1}^n (x_i - \hat{\mu})^2\end{aligned}$$



Estimating the parameters of a d -dim Gaussian Model from observations

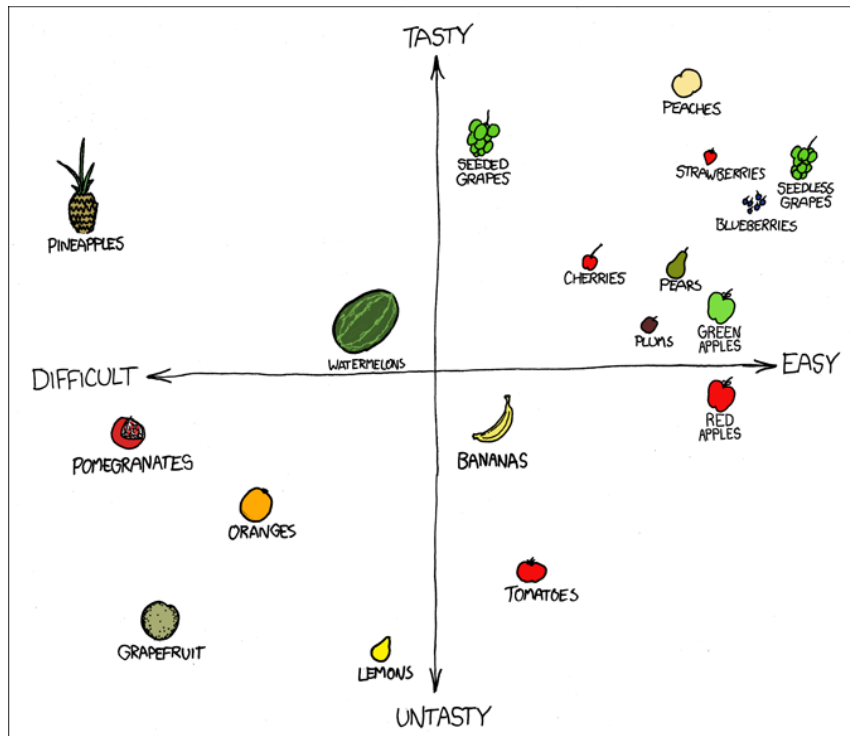
- ◆ Exact same approach as for the 1-dim case, but using

- d -dim vectors $\vec{x}_1, \vec{x}_2, \dots, \vec{x}_n$ as observation
- a d -dim vector as mean $\vec{\mu}$ and
- a $d \times d$ -dim covariance matrix $\hat{\Sigma}$
- and the multivariate Gaussian distribution $\mathcal{N}(\vec{x}; \vec{\mu}, \Sigma) = \frac{1}{\sqrt{(2\pi)^d \det(\Sigma)}} \exp \left[-\frac{1}{2} (\vec{x} - \vec{\mu})^T \Sigma^{-1} (\vec{x} - \vec{\mu}) \right]$

leads to

$$\vec{\hat{\mu}} = \frac{1}{n} \sum_{i=1}^n \vec{x}_i$$

$$\hat{\Sigma} = \frac{1}{n} \sum_{i=1}^n (\vec{x}_i - \vec{\hat{\mu}})^T (\vec{x}_i - \vec{\hat{\mu}})$$



source: <https://xkcd.com/388/>

Further Reading:

EM and proof it converges

https://en.wikipedia.org/wiki/Expectation%E2%80%93maximization_algorithm

<https://www.youtube.com/watch?v=ZZGTuAkF-Hw>

MLE:

<https://www.youtube.com/watch?v=tV3RIs3NGec>

<https://www.youtube.com/watch?v=LCWsld9ZCB4>

EM:

<https://www.youtube.com/watch?v=qMTuMa86NzU>

https://www.youtube.com/watch?v=REypj2sy_5U&index=1&list=PLBv09BD7ez_4e9LtmK626Evn1ion6ynrt



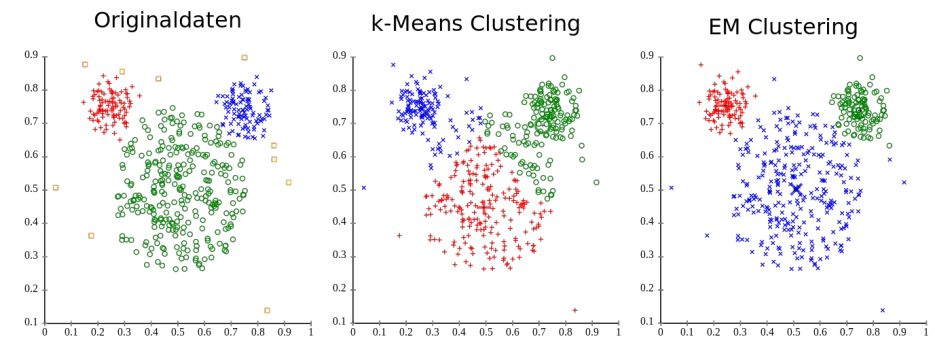
Key Takeaways

◆ GMM – Gaussian Mixture Models

- Flexible and frequently used class of probability distributions
- Explain observations using latent (hidden) groupings of data

◆ EM Algorithm (Expectation Maximization)

- Generalization of k -means
- Computes soft cluster membership estimations r_{ic}
- Estimates latent parameters of the Gaussians
- Various ways to choose number of clusters k



source: https://en.wikipedia.org/wiki/K-means_clustering